Natural language processing

Unit 4 · reference sheet · AI/ML in 35 topics

Text to IDs

Token ≠ word. English ≈ 4 chars/token ≈ 0.75 words.
Code ≈ 1.7 tokens/word; base64 far worse.
BPE: start from characters, merge the most frequent adjacent pair.

Embeddings

$E \in \mathbb{R}^{V \times d}$; token $i \mapsto E[i]$ (a lookup, not a matmul)
$\cos(u, v) = \dfrac{u \cdot v}{\|u\|\|v\|}$ length is not meaning
Static (Word2Vec) vs contextual (transformer): one vector per *occurrence*

Recurrence

$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$ state is fixed size
BPTT = product of T Jacobians: $0.9^{20} = 0.12$
LSTM: $c_t = f \odot c_{t-1} + i \odot g$, $h_t = o \odot \tanh(c_t)$
Additive cell path ⇒ gradient near identity.

Seq2seq

encoder $x_{1:n} \to h_n$; decoder $p(y_t \mid h_n, y_{<t})$
Teacher forcing at train, free-running at test ⇒ exposure bias
Bottleneck: 600 tokens into 512 floats = 0.85 floats/token.

Gotchas

- Truncating by characters can split a token.
- Attention weights are not causal explanations.
- LSTM mitigates decay; it does not remove the bottleneck.

This unit at a glance

#	Topic	You can
15	Tokenization	explain vocab, subwords, OOV, and why token count word count
16	Embeddings	explain the distributional hypothesis and compute cosine similarity
17	RNN → LSTM	explain the fixed-vector bottleneck and how gates address it
18	Seq2seq & encoder–decoder	explain translation as encode-then-decode and state the alignment problem

If you see this → do that

Signal	Move
Vocabulary too large	subwords (BPE)
Rare or unseen words	BPE, never UNK
One vector per word fails	contextual encoder
Long input	attention
Must generate text	decoder

Numbers worth memorising

English: 1 token ≈ 4 chars ≈ 0.75 words
1k words ≈ 1.3k tokens; code ≈ 1.7k
512 floats for 600 tokens = 0.85 floats/token
$0.9^{20} = 0.12$ $0.99^{100} = 0.366$